

Final Report: Modeling and Comparing Basic Scheduling Algorithms

1. Introduction and Background

- Operating systems are responsible for managing hardware and software resources.
 - One of the most important responsibilities of an operating system is **process management**.
 - CPU scheduling is a key part of process management.
 - CPU scheduling decides which process gets access to the CPU at a specific time.
 - Without CPU scheduling, processes would not be executed in an organized and efficient way.
 - This project focuses on building a simplified **CPU scheduling simulator**.
 - The goal of the simulator is to show how different scheduling algorithms affect:
 - Execution order
 - Waiting time
 - Turnaround time
 - Overall system performance
 - The project compares three basic CPU scheduling algorithms:
 - **First Come First Served**
 - **Shortest Job First**
 - **Round Robin**
 - This project is connected to real operating systems because systems such as Linux and Windows also use scheduling concepts to manage multiple processes.
 - The project does not try to build a full operating system.
 - Instead, it focuses on one important operating system subsystem: **CPU scheduling**.
-

2. Design and Implementation Details

- The project was designed as a simple CPU scheduling simulator.
- The simulator takes process information as input.
- Each process contains:
 - Process ID
 - Arrival time
 - Burst time
- For Round Robin scheduling, the program also uses:
 - Time quantum
- The system follows a modular design.
- The main modules are:
 - Input Module
 - Process Management Module

- Scheduling Module
 - Performance Calculation Module
 - Output Module
- The architecture from Phase 1 shows the flow of the system from input, to process organization, to scheduling, to performance calculation, and finally to output display.

Input Module

- This module collects the process data.
- The user enters process details such as:
 - Process ID
 - Arrival time
 - Burst time
 - Time quantum for Round Robin
- The input data is then sent to the process management part of the program.

Process Management Module

- This module stores and organizes all process information.
- It keeps track of each process before and during scheduling.
- It helps the scheduling algorithms access the correct process data.

Scheduling Module

- This is the main part of the simulator.
- It applies the selected CPU scheduling algorithm.
- The implemented algorithms are:

First Come First Served

- FCFS schedules processes based on their arrival order.
- The process that arrives first is executed first.
- It is easy to implement and understand.
- However, it may cause long waiting times if a long process arrives before shorter ones.

Shortest Job First

- SJF selects the process with the shortest burst time.
- It can reduce average waiting time.
- In the sample test case, SJF gave better results than FCFS and Round Robin.
- However, it may not be fair to longer processes because they can wait longer.

Round Robin

- Round Robin gives each process a fixed amount of CPU time.
- This fixed time is called the time quantum.

- In the project's sample test case, the time quantum was set to 2.
- If a process does not finish during its time quantum, it goes back to the queue.
- Round Robin is fair because each process gets a turn.
- However, it can create higher waiting and turnaround times depending on the time quantum.

Performance Calculation Module

- After scheduling is complete, the program calculates:
 - Waiting time
 - Turnaround time
 - Average waiting time
 - Average turnaround time
 - Execution order
- These results are used to compare the algorithms.

Output Module

- This module displays the final results.
- The output shows:
 - The order in which processes were executed
 - Waiting time for each process
 - Turnaround time for each process
 - Average waiting time
 - Average turnaround time
- The output allows the user to clearly compare the behavior of FCFS, SJF, and Round Robin.

3. Final Results and Comparison

- The project used a sample process set to test the algorithms.
- The sample processes were:

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	8
P4	3	6

- For **FCFS**, the execution order was:
 - $P1 \rightarrow P2 \rightarrow P3 \rightarrow P4$
 - FCFS produced:
 - Average waiting time: **5.75**
 - Average turnaround time: **11.25**
 - For **SJF**, the execution order was:
 - $P1 \rightarrow P2 \rightarrow P4 \rightarrow P3$
 - SJF produced:
 - Average waiting time: **5.25**
 - Average turnaround time: **10.75**
 - For **Round Robin**, the time quantum was set to 2.
 - Round Robin produced:
 - Average waiting time: **9.75**
 - Average turnaround time: **15.25**
 - Based on these results, SJF performed best for this test case.
 - FCFS was simple, but some processes waited longer.
 - Round Robin was fairer because each process received CPU time in turns, but it produced higher waiting and turnaround times in this example.
-

4. Challenges Encountered and How They Were Addressed

- Round Robin was more difficult than FCFS and SJF.
 - This is because processes may not finish in one turn and must return to the queue.
 - This was addressed by tracking remaining burst time for each process.
 - Keeping the project simple was also a challenge.
 - We avoided unnecessary advanced features and focused only on the required scheduling algorithms.
 - Testing was important because small calculation errors could affect the final results.
 - This was handled by checking the output manually using sample test cases.
-

5. Trade-offs in Design Choices

- The project uses a simplified simulation instead of a real operating system scheduler.
 - This made the project easier to understand and complete.
 - However, it does not include all the complexity of real OS scheduling.
- The project focuses on three basic algorithms only:
 - FCFS
 - SJF
 - Round Robin
- This keeps the project manageable.

- More advanced algorithms such as priority scheduling, multilevel queue scheduling, or real-time scheduling were not included.
 - The simulator focuses on basic performance metrics:
 - Waiting time
 - Turnaround time
 - Execution order
 - This makes the comparison clear.
 - However, it does not measure more advanced metrics such as CPU utilization, throughput, or response time.
 - The project does not include a graphical user interface.
 - A simple console-based program is easier to implement and test.
 - This also allows us to focus more on scheduling logic instead of interface design.
 - Round Robin uses a fixed time quantum.
 - This makes implementation simpler.
-

6. Quality and Structure of the Code

- The code is organized around the main parts of the simulator.
- The main responsibilities of the code are separated clearly:
 - Input handling
 - Process storage
 - Scheduling algorithm execution
 - Performance calculation
 - Result display
- The code is written to be understandable and easy to follow.
- Variable names are chosen to match their purpose, such as:
 - arrival time
 - burst time
 - waiting time
 - turnaround time
 - remaining time
- Comments are included to explain important parts of the program.
- Comments are especially useful in:
 - FCFS implementation
 - SJF process selection
 - Round Robin loop
 - Waiting time calculation
 - Turnaround time calculation
- Each algorithm is implemented in a direct and readable way.
- The structure makes it easier to test each algorithm separately.
- The program can be improved in the future by adding:
 - Priority scheduling
 - More test cases
 - File input/output

- A graphical interface
 - More performance metrics
-

7. Conclusion

- The final project successfully implements a simplified CPU scheduling simulator.
- The simulator compares FCFS, SJF, and Round Robin scheduling algorithms.
- The project shows how different scheduling decisions affect process execution.
- The results show that SJF performed best for the selected sample data.
- FCFS was the simplest algorithm but was not always the most efficient.
- Round Robin was fairer but produced higher average waiting and turnaround times in the sample test.
- Through this project, the team gained a better understanding of:
 - CPU scheduling
 - Process execution
 - Waiting time
 - Turnaround time
 - Algorithm comparison
- Overall, the project connects operating system theory with practical implementation.
- It meets the Phase 2 requirement of creating a working solution, explaining the design, discussing challenges, analyzing trade-offs, and describing the quality and structure of the code.